

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

ANAGHA AVINASH (1BM24CS352)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Anagha Avinash(1BM24CS352), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. → FCFS → SJF (pre-emptive & Non-pre-emptive) → Priority (pre-emptive & Non-pre-emptive) → Round Robin	1-15
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	16-19
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	20-22
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	23-28
5.	Write a C program to simulate. a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	29-37
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	38-43
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	44-50

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→ FCFS

→ SJF (pre-emptive & Non-preemptive)

→ Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS:

```
#include<stdio.h>
#include<conio.h>
int main(){
    int n;
    int at[20],bt[20],ct[20],tat[20],wt[20];
    int t,ti;
    float tatsum=0,wtsum=0;
    printf("enter number of process id");
    scanf("%d",&n);
    printf("enter arrival time:\n");
    for(int i=0;i<n;i++){
        scanf("%d",&at[i]);
    }
    printf("enter burst time:");
    for(int i=0;i<n;i++){
        scanf("%d",&bt[i]);
    }
    ct[0]=bt[0];

    for(int i=1;i<n;i++){
        ct[i]=ct[i-1]+bt[i];
    }
    for(int i=0;i<n;i++)
    {
```

```

tat[i]=ct[i]-at[i];
tatsum+=tat[i];
}
for(int i=0;i<n;i++){
    wt[i]=tat[i]-bt[i];
    wtsum+=wt[i];

}
printf("process id\tat\tbt\tct\ttat\twt\t\n");
for(int i=0;i<n;i++){
    printf("%d \t %d \t %d \t %d \t %d \t %d\n",i,at[i],bt[i],ct[i],tat[i],wt[i]);
}
printf("average tat=%f",tatsum/n);
printf("average waiting time:%f",wtsum/n);
}

```

Output:

```

Enter number of processes: 3
Enter Arrival Time and Burst Time:
1
5
2
3
3
4

P      AT      BT      CT      TAT      WT      RT
P1      1      5      6      5      0      0
P2      2      3      9      7      4      4
P3      3      4     13     10      6      6

Average Waiting Time = 3.33
Average Turnaround Time = 7.33
Average Response Time = 3.33

```

SJF(Non-preemptive):

```
#include<stdio.h>
```

```

int main()
{
    int p[20],at[20],bt[20],ct[20],tat[20],wt[20];
    int n,completed=0,time=0,min,idx;

```

```

int visited[20]={0};
float tatsum=0,wtsum=0;

printf("Enter number of processes: ");
scanf("%d",&n);

printf("Enter Process IDs:\n");
for(int i=0;i<n;i++)
    scanf("%d",&p[i]);

printf("Enter Arrival Time:\n");
for(int i=0;i<n;i++)
    scanf("%d",&at[i]);

printf("Enter Burst Time:\n");
for(int i=0;i<n;i++)
    scanf("%d",&bt[i]);

while(completed<n)
{
    min=9999;
    idx=-1;

    for(int i=0;i<n;i++)
    {
        if(at[i]<=time && visited[i]==0 && bt[i]<min)
        {
            min=bt[i];
            idx=i;
        }
    }

    if(idx!=-1)
    {

```

```

        time += bt[idx];
        ct[idx]=time;
        visited[idx]=1;
        completed++;
    }
    else
    {
        time++;
    }
}

for(int i=0;i<n;i++)
{
    tat[i]=ct[i]-at[i];
    tatsum+=tat[i];

    wt[i]=tat[i]-bt[i];
    wtsum+=wt[i];
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(int i=0;i<n;i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);

printf("\nAverage TAT = %.2f",tatsum/n);
printf("\nAverage WT = %.2f",wtsum/n);

return 0;
}

```


Output:

```
Enter number of processes: 3
Enter Arrival Time and Burst Time:
1
5
2
3
3
4

P      AT      BT      CT      TAT      WT      RT
P1      1       5       6       5       0       0
P2      2       3       9       7       4       4
P3      3       4      13      10       6       6

Average Waiting Time = 3.33
Average Turnaround Time = 7.33
Average Response Time = 3.33
```

SJF(Pre-emptive):

```
#include<stdio.h>
```

```
int main()
{
    int n;
    int p[20],at[20],bt[20],rt[20];
    int ct[20],tat[20],wt[20],rtm[20];
    int completed=0,time=0,min,idx;
    float tatsum=0,wtsum=0,rtsum=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Process ID:\n");
    for(int i=0;i<n;i++)
        scanf("%d",&p[i]);

    printf("Enter Arrival Time:\n");
    for(int i=0;i<n;i++)
```

```

scanf("%d",&at[i]);

printf("Enter Burst Time:\n");
for(int i=0;i<n;i++){
    scanf("%d",&bt[i]);
    rt[i]=bt[i];    // remaining time
    rtm[i]=-1;      // response time marker
}

while(completed<n)
{
    min=9999;
    idx=-1;

    for(int i=0;i<n;i++)
    {
        if(at[i]<=time && rt[i]>0 && rt[i]<min)
        {
            min=rt[i];
            idx=i;
        }
    }

    if(idx!=-1)
    {
        if(rtm[idx]==-1)
            rtm[idx]=time-at[idx]; // response time

        rt[idx]--;
        time++;

        if(rt[idx]==0)
        {

```

```

        ct[idx]=time;
        completed++;
    }
}
else
{
    time++;
}
}

for(int i=0;i<n;i++)
{
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];

    tatsum+=tat[i];
    wtsum+=wt[i];
    rtsum+=rtm[i];
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(int i=0;i<n;i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i],at[i],bt[i],ct[i],tat[i],wt[i],rtm[i]);

printf("\nAverage TAT = %.2f",tatsum/n);
printf("\nAverage WT = %.2f",wtsum/n);
printf("\nAverage RT = %.2f",rtsum/n);

return 0;
}

```

Output:

```
Enter number of processes: 3
Enter AT and BT:
1
5
2
3
3
4

P      AT      BT      CT      TAT      WT      RT
P1     1       5       9       8       3       0
P2     2       3       5       3       0       0
P3     3       4      13      10       6       6

Average Waiting Time = 3.00
Average Turnaround Time = 7.00
Average Response Time = 2.00
```

Priority(Pre-emptive):

```
#include <stdio.h>
```

```
struct Process {
    int pid, bt, at, pr;
    int rt;
    int wt, tat, ct;
    int completed;
};
```

```
int main() {
    int n, time = 0, completed = 0;
    struct Process p[20];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
```

```

printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i+1);
scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);

p[i].pid = i + 1;
p[i].rt = p[i].bt;
p[i].completed = 0;
}

while(completed < n) {
    int idx = -1;
    int highest = 9999;

    for(int i = 0; i < n; i++) {
        if(p[i].at <= time && p[i].rt > 0) {
            if(p[i].pr < highest) {
                highest = p[i].pr;
                idx = i;
            }
        }
    }

    if(idx != -1) {
        p[idx].rt--;
        time++;

        if(p[idx].rt == 0) {
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            p[idx].completed = 1;
            completed++;
        }
    }
    else {
        time++;
    }
}

```

```

    }
}

printf("\nPID\tAT\tBT\tPR\tWT\tTAT\n");

for(int i = 0; i < n; i++) {
    avgwt += p[i].wt;
    avgtat += p[i].tat;

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].wt, p[i].tat);
}

printf("\nAverage Waiting Time = %.2f", avgwt/n);
printf("\nAverage Turnaround Time = %.2f\n", avgtat/n);

return 0;
}

```

Output:

```

Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 5 2
Enter Arrival Time, Burst Time and Priority for P2: 1 3 1
Enter Arrival Time, Burst Time and Priority for P3: 2 4 3

PID      AT      BT      PR      WT      TAT
P1        0        5        2        3        8
P2        1        3        1        0        3
P3        2        4        3        6       10

Average Waiting Time = 3.00
Average Turnaround Time = 7.00

```

Priority(Non-Preemptive):

```
#include <stdio.h>

struct Process {
    int pid, bt, at, pr;
    int wt, tat, ct, completed;
};

int main() {
    int n, time = 0, completed = 0;
    struct Process p[20];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i+1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].pid = i+1;
        p[i].completed = 0;
    }

    while(completed < n) {
        int idx = -1;
        int highest = 9999;

        for(int i = 0; i < n; i++) {
            if(p[i].at <= time && p[i].completed == 0) {
                if(p[i].pr < highest) {
                    highest = p[i].pr;
                    idx = i;
                }
            }
        }
    }
}
```

```

    }
}

if(idx != -1) {
    time += p[idx].bt;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].completed = 1;
    completed++;
} else {
    time++;
}
}

printf("\nPID\tAT\tBT\tPR\tWT\tTAT\n");

for(int i = 0; i < n; i++) {
    avgwt += p[i].wt;
    avgtat += p[i].tat;

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].wt, p[i].tat);
}

printf("\nAverage Waiting Time = %.2f", avgwt/n);
printf("\nAverage Turnaround Time = %.2f\n", avgtat/n);

return 0;
}

```


Output:

```
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 5 2
Enter Arrival Time, Burst Time and Priority for P2: 1 3 1
Enter Arrival Time, Burst Time and Priority for P3: 2 4 3

PID      AT      BT      PR      WT      TAT
P1        0        5        2        0        5
P2        1        3        1        4        7
P3        2        4        3        6       10

Average Waiting Time = 3.33
Average Turnaround Time = 7.33
```

Round Robin:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, time = 0, remain;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int AT[n], BT[n], RT[n], CT[n], WT[n], TAT[n];
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time for P%d: ", i+1);
```

```
        scanf("%d", &AT[i]);
```

```
        printf("Enter Burst Time for P%d: ", i+1);
```

```
        scanf("%d", &BT[i]);
```

```
        RT[i] = BT[i];
```

```
    }
```

```
    int tq;
```

```
    printf("Enter Time Quantum: ");
```

```
    scanf("%d", &tq);
```

```
    remain = n;
```

```

int flag = 0;

while(remain > 0) {
    flag = 0;
    for(i = 0; i < n; i++) {
        if(RT[i] > 0 && AT[i] <= time) {
            flag = 1;
            if(RT[i] <= tq) {
                time += RT[i];
                RT[i] = 0;
                CT[i] = time;
                remain--;
            } else {
                time += tq;
                RT[i] -= tq;
            }
        }
    }
    if(flag == 0) time++;
}

float totalWT = 0, totalTAT = 0;

printf("\nPID\tAT\tBT\tCT\tWT\tTAT\n");
for(i = 0; i < n; i++) {
    TAT[i] = CT[i] - AT[i];
    WT[i] = TAT[i] - BT[i];
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, AT[i], BT[i], CT[i], WT[i], TAT[i]);
    totalWT += WT[i];
    totalTAT += TAT[i];
}

printf("\nAverage WT = %.2f", totalWT / n);
printf("\nAverage TAT = %.2f\n", totalTAT / n);

```

```
    return 0;  
}
```

Output:

```
Enter number of processes: 3  
Enter Arrival Time for P1: 1  
Enter Burst Time for P1: 7  
Enter Arrival Time for P2: 2  
Enter Burst Time for P2: 8  
Enter Arrival Time for P3: 3  
Enter Burst Time for P3: 12  
Enter Time Quantum: 2  
  
PID      AT      BT      CT      WT      TAT  
P1        1        7      20      12      19  
P2        2        8      22      12      20  
P3        3       12      28      13      25  
  
Average WT = 12.33  
Average TAT = 21.33
```

Program -2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

typedef struct {
    int pid;
    int arrival;
    int burst;
    int remaining;
    char type[10]; // "system" or "user"
} Process;

typedef struct {
    Process queue[MAX];
    int front, rear;
} Queue;

void enqueue(Queue *q, Process p) {
    if(q->rear == MAX-1) return;
    q->queue[++q->rear] = p;
    if(q->front == -1) q->front = 0;
}

Process dequeue(Queue *q) {
    Process p = q->queue[q->front++];
    if(q->front > q->rear) {
        q->front = q->rear = -1;
    }
    return p;
}
```

```

}

int isEmpty(Queue *q) {
    return q->front == -1;
}

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    Process plist[n];
    for(int i=0; i<n; i++) {
        printf("Enter Process ID: ");
        scanf("%d", &plist[i].pid);
        printf("Enter Arrival Time: ");
        scanf("%d", &plist[i].arrival);
        printf("Enter Burst Time: ");
        scanf("%d", &plist[i].burst);
        printf("Enter Type (system/user): ");
        scanf("%s", plist[i].type);
        plist[i].remaining = plist[i].burst;
    }

    for(int i=0; i<n-1; i++) {
        for(int j=i+1; j<n; j++) {
            if(plist[i].arrival > plist[j].arrival) {
                Process temp = plist[i];
                plist[i] = plist[j];
                plist[j] = temp;
            }
        }
    }

    Queue systemQ = {.front=-1, .rear=-1};
    Queue userQ = {.front=-1, .rear=-1};

```

```

int current_time = 0, completed = 0, i = 0;
Process *current_process = NULL;

while(completed < n) {
    while(i < n && plist[i].arrival <= current_time) {
        if(strcmp(plist[i].type, "system") == 0)
            enqueue(&systemQ, plist[i]);
        else
            enqueue(&userQ, plist[i]);
        i++;
    }

    if(current_process != NULL) {
        if(strcmp(current_process->type, "user") == 0 && !isEmpty(&systemQ)) {
            enqueue(&userQ, *current_process);
            current_process = NULL;
        }
    }

    if(current_process == NULL) {
        if(!isEmpty(&systemQ)) {
            Process p = dequeue(&systemQ);
            current_process = (Process*)malloc(sizeof(Process));
            *current_process = p;
        } else if(!isEmpty(&userQ)) {
            Process p = dequeue(&userQ);
            current_process = (Process*)malloc(sizeof(Process));
            *current_process = p;
        } else {
            current_time++;
            continue;
        }
    }

    current_process->remaining--;
    current_time++;
}

```

```

if(current_process->remaining == 0) {
    printf("Process %d (%s) completed at time %d\n",
        current_process->pid, current_process->type, current_time);
    completed++;
    free(current_process);
    current_process = NULL;
}
}

return 0;
}

```

Output:

```

Enter number of processes: 3
Enter Process ID: 1
Enter Arrival Time: 1
Enter Burst Time: 7
Enter Type (system/user): system
Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 11
Enter Type (system/user): user
Enter Process ID: 3
Enter Arrival Time: 3
Enter Burst Time: 10
Enter Type (system/user): system
Process 1 (system) completed at time 8
Process 3 (system) completed at time 18
Process 2 (user) completed at time 29

```

Program -3

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a)Rate- Monotonic
- b)Earliest-deadline First
- c)Proportional scheduling

```
#include <stdio.h>
```

```
#include <math.h>
```

```
typedef struct {
```

```
    int id;
```

```
    int burst;
```

```
    int deadline;
```

```
    int period;
```

```
    int completion;
```

```
    int waiting;
```

```
    int tat;
```

```
} Process;
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    Process p[n];
```

```
    printf("Enter process details:\n");
```

```
    for(int i=0; i<n; i++) {
```

```
        p[i].id = i;
```

```
        printf("Process %d:\n", i);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &p[i].burst);
```

```
        printf("Deadline (for EDF): ");
```

```
        scanf("%d", &p[i].deadline);
```

```
        printf("Period (for RMS): ");
```



```

    scanf("%d", &p[i].period);
}

// CPU Utilization
double utilization = 0.0;
for(int i=0; i<n; i++) {
    utilization += (double)p[i].burst / p[i].period;
}

// EDF schedulability
printf("\nEarliest Deadline First (EDF) Scheduling\n");
printf("CPU Utilization:\n%.2f\n", utilization);
if(utilization <= 1.0)
    printf("Schedulable (Utilization <= 1)\n");
else
    printf("NOT Schedulable (Utilization > 1)\n");

printf("ID\tBF\tDeadline\tCT\tWT\tTAT\n");
for(int i=0; i<n; i++) {
    p[i].completion = p[i].burst + (i==0?0:p[i-1].completion);
    p[i].tat = p[i].completion; // since arrival=0
    p[i].waiting = p[i].tat - p[i].burst;
    printf("%d\t%d\t%d\t\t%d\t%d\t%d\n",
        p[i].id, p[i].burst, p[i].deadline,
        p[i].completion, p[i].waiting, p[i].tat);
}

// RMS schedulability
double bound = n * (pow(2.0, 1.0/n) - 1.0);
printf("\nRate Monotonic Scheduling (RMS)\n");
printf("CPU Utilization: %.2f\n", utilization);
printf("RM Bound: %.4f\n", bound);
if(utilization <= bound)

```

```

    printf("Schedulable (Utilization <= RM Bound)\n");
else
    printf("NOT Schedulable (Utilization > RM Bound)\n");

printf("ID\tBF\tPeriod\tCT\tWT\tTAT\n");
for(int i=0; i<n; i++) {
    p[i].completion = p[i].burst + (i==0?0:p[i-1].completion);
    p[i].tat = p[i].completion;
    p[i].waiting = p[i].tat - p[i].burst;
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id, p[i].burst, p[i].period,
        p[i].completion, p[i].waiting, p[i].tat);
}
return 0;
}

```

Output:

```

Process 1:
Burst Time: 3
Deadline (for EDF): 5
Period (for RMS): 7
Process 2:
Burst Time: 2
Deadline (for EDF): 9
Period (for RMS): 10

Earliest Deadline First (EDF) Scheduling
CPU Utilization:
0.72
Schedulable (Utilization <= 1)

```

ID	BF	Deadline	CT	WT	TAT
0	2	7	2	0	2
1	3	5	5	2	5
2	2	9	7	5	7

```

Rate Monotonic Scheduling (RMS)
CPU Utilization: 0.72
RM Bound: 0.7798
Schedulable (Utilization <= RM Bound)

```

ID	BF	Period	CT	WT	TAT
0	2	22	2	0	2
1	3	7	5	2	5
2	2	10	7	5	7

Program -4

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem.

Producer-Consumer problem using semaphores

```
#include <stdio.h>

#include <stdlib.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0;

int out = 0;


int empty = BUFFER_SIZE;
int full = 0;


void producer() {
    if (empty == 0) {
        printf("\nBuffer is full! Cannot produce.\n");
        return;
    }
    int item;
    printf("Enter item to produce: ");
    scanf("%d", &item);
    buffer[in] = item;
    printf("Produced: %d at index %d\n", item, in);
    in = (in + 1) % BUFFER_SIZE;
    empty--;
    full++;
}


void consume() {
```

```

if (full == 0) {
    printf("\nBuffer is empty! Cannot consume.\n");
    return;
}

int item = buffer[out];
printf("Consumed: %d from index %d\n", item, out);

out = (out + 1) % BUFFER_SIZE;
full--;
empty++;
}

int main() {
    int choice;
    printf("Buffer Size: %d\n", BUFFER_SIZE);

    while (1) {
        printf("\n1. Produce\n2. Consume\n3. Exit\n");
        printf("Current State: [Full slots: %d | Empty slots: %d]\n", full, empty);
        printf("Enter choice: ");
        if (scanf("%d", &choice) != 1) break;
        switch (choice) {
            case 1:
                produce();
                break;
            case 2:
                consume();
                break;
            case 3:
                printf("Exiting...\n");

```

```

        exit(0);

    default:

        printf("Invalid choice!\n");

    }}

    return 0;
}

```

Output:

```

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 0 | Empty slots: 5]
Enter choice: 1
Enter item to produce: 10
Produced: 10 at index 0

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 1
Enter item to produce: 20
Produced: 20 at index 1

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 2 | Empty slots: 3]
Enter choice: 2
Consumed: 10 from index 0

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 2
Consumed: 20 from index 1

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 0 | Empty slots: 5]
Enter choice: 3
Exiting...

```

Dining Philosophers problem.

```

#include<stdio.h>

#include<pthread.h>

#include<unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N]; // One mutex per fork

```

```

pthread_t philosophers[N];
void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id; // left fork index
    int right = (id + 1) % N; // right fork index
    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1); // Thinking
        // To avoid deadlock, philosophers with even id pick left then right,
        // odd id pick right then left.
        if (id % 2 == 0)
        {
            // Pick up left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
            // Pick up right fork

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); }
        else
        { // Pick up right fork
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); // Pick up left
            fork pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
        // Eating
        printf("Philosopher %d is eating.\n", id);
        sleep(2); // Put down forks
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
    }
}

```

```

printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
return NULL;
}

```

```

int main() {
int i;
int ids[N]; // Initialize forks (mutexes)
for (i = 0; i < N; i++)
{ pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
// Create philosopher threads
for (i = 0; i < N; i++)

{
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
} // Join threads (will never happen here, but good practice)
for (i = 0; i < N; i++)
{
pthread_join(philosophers[i], NULL); }
// Destroy mutexes (unreachable here)
for (i = 0; i < N; i++)
{
pthread_mutex_destroy(&forks[i]);
}
return 0;
}

```

Output:

```
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
Consumed: 10 from buffer[0]
Produced: 15 at buffer[0]
Consumed: 11 from buffer[1]
Produced: 16 at buffer[1]
Consumed: 12 from buffer[2]
Produced: 17 at buffer[2]
Consumed: 13 from buffer[3]
Produced: 18 at buffer[3]
```


Program -5

Write a C program to simulate.

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

Bankers algorithm deadlock avoidance:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], need[n][m];
```

```
    int avail[m];
```

```
    printf("\nEnter Allocation Matrix:\n");
```

```
    for(int i = 0; i < n; i++)
```

```
        for(int j = 0; j < m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    printf("\nEnter Maximum Matrix:\n");
```

```
    for(int i = 0; i < n; i++)
```

```
        for(int j = 0; j < m; j++)
```

```
            scanf("%d", &max[i][j]);
```

```

printf("\nEnter Available Resources:\n");
for(int i = 0; i < m; i++)
    scanf("%d", &avail[i]);

for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

printf("\nNeed Matrix:\n");

for(int i = 0; i < n; i++)
{
    for(int j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

int process;
int request[m];

printf("\nEnter process number requesting resources: ");
scanf("%d", &process);

printf("Enter request for each resource:\n");

for(int i = 0; i < m; i++)
    scanf("%d", &request[i]);

```

```

for(int i = 0; i < m; i++)
{
    if(request[i] > need[process][i])
    {
        printf("\nError: Process has exceeded maximum claim.\n");
        return 0;
    }
}

```

```

for(int i = 0; i < m; i++)
{
    if(request[i] > avail[i])
    {
        printf("\nResources are not available. Process must wait.\n");
        return 0;
    }
}

```

```

for(int i = 0; i < m; i++)
{
    avail[i] -= request[i];
    alloc[process][i] += request[i];
    need[process][i] -= request[i];
}

```

```

int finish[n], safeSeq[n];

```

```

for(int i = 0; i < n; i++)
    finish[i] = 0;

```

```

int work[m];

```

```

for(int i = 0; i < m; i++)

```

```

    work[i] = avail[i];

int count = 0;

while(count < n)
{
    int found = 0;

    for(int p = 0; p < n; p++)
    {
        if(finish[p] == 0)
        {
            int j;

            for(j = 0; j < m; j++)
            {
                if(need[p][j] > work[j])
                    break;
            }

            if(j == m)
            {
                for(int k = 0; k < m; k++)
                    work[k] += alloc[p][k];

                safeSeq[count++] = p;
                finish[p] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
    {

```

```

        printf("\nRequest cannot be granted.");
        printf("\nSystem will enter unsafe state.\n");
        return 0;
    }
}

```

```

printf("\nRequest can be granted.\n");
printf("System is in SAFE state.\n");

```

```

printf("Safe Sequence: ");

```

```

for(int i = 0; i < n; i++)
    printf("P%d ", safeSeq[i]);

```

```

printf("\n");

```

```

return 0;

```

```

}

```

Output:

```

Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

Enter process number requesting resources: 1
Enter request for each resource:
1 0 2

Request can be granted.
System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2

```

Deadlock Detection:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], request[n][m], avail[m];
```

```
    printf("\nEnter Allocation Matrix:\n");
```

```
    for(int i = 0; i < n; i++)
```

```
        for(int j = 0; j < m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    printf("\nEnter Request Matrix:\n");
```

```
    for(int i = 0; i < n; i++)
```

```
        for(int j = 0; j < m; j++)
```

```
            scanf("%d", &request[i][j]);
```

```
    printf("\nEnter Available Resources:\n");
```

```
    for(int i = 0; i < m; i++)
```

```
        scanf("%d", &avail[i]);
```

```
    int finish[n];
```

```
    for(int i = 0; i < n; i++)
```

```
{
```

```

int flag = 0;

for(int j = 0; j < m; j++)
{
    if(alloc[i][j] != 0)
    {
        flag = 1;
        break;
    }
}

finish[i] = (flag == 0) ? 1 : 0;
}

int work[m];

for(int i = 0; i < m; i++)
    work[i] = avail[i];

while(1)
{
    int found = 0;

    for(int i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int j;

            for(j = 0; j < m; j++)
            {
                if(request[i][j] > work[j])
                    break;
            }
        }
    }
}

```

```

    }

    if(j == m)
    {
        for(int k = 0; k < m; k++)
            work[k] += alloc[i][k];

        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
    break;
}

int deadlock = 0;

printf("\nDeadlocked Processes: ");

for(int i = 0; i < n; i++)
{
    if(finish[i] == 0)
    {
        printf("P%d ", i);
        deadlock = 1;
    }
}

if(deadlock == 0)
    printf("None");

```



```
printf("\n");

return 0;
}
```

Output:

```
Enter number of processes: 4
Enter number of resources: 3

Enter Allocation Matrix:
1 0 2
2 1 1
1 0 3
1 2 2

Enter Request Matrix:
0 0 1
1 0 2
0 0 0
3 3 0

Enter Available Resources:
0 0 0

Deadlocked Processes: P3
```

Program -6

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

```
#include <stdio.h>
```

```
void firstFit(int blockSize[], int blocks, int processSize[], int processes) {
```

```
    int allocation[processes];
```

```
    for (int i = 0; i < processes; i++)
```

```
        allocation[i] = -1;
```

```
    int used[blocks];
```

```
    for (int i = 0; i < blocks; i++)
```

```
        used[i] = 0;
```

```
    for (int i = 0; i < processes; i++) {
```

```
        for (int j = 0; j < blocks; j++) {
```

```
            // block must be unused
```

```
            if (!used[j] && blockSize[j] >= processSize[i]) {
```

```
                allocation[i] = j;
```

```
                used[j] = 1; // mark block as used
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("\n--- First Fit ---\n");
```

```
    printf("Process No.\tProcess Size\tBlock No.\n");
```

```

for (int i = 0; i < processes; i++) {
    printf("%d\t%d\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

void bestFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[processes];

    for (int i = 0; i < processes; i++)
        allocation[i] = -1;

    int used[blocks];
    for (int i = 0; i < blocks; i++)
        used[i] = 0;

    for (int i = 0; i < processes; i++) {
        int bestIdx = -1;

        for (int j = 0; j < blocks; j++) {

            if (!used[j] && blockSize[j] >= processSize[i]) {

                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
    }
}

```

```

        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            used[bestIdx] = 1;
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < processes; i++) {
        printf("%d\t%d\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[processes];

    for (int i = 0; i < processes; i++)
        allocation[i] = -1;

    int used[blocks];
    for (int i = 0; i < blocks; i++)
        used[i] = 0;

    for (int i = 0; i < processes; i++) {
        int worstIdx = -1;

```

```

for (int j = 0; j < blocks; j++) {

    if (!used[j] && blockSize[j] >= processSize[i]) {

        if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
            worstIdx = j;
        }
    }
}

if (worstIdx != -1) {
    allocation[i] = worstIdx;
    used[worstIdx] = 1;
}
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < processes; i++) {
    printf("%d\t%d\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main() {
    int blocks, processes;
    printf("Enter number of memory blocks: ");
    scanf("%d", &blocks);

```

```

int blockSize[blocks];

printf("Enter sizes of memory blocks:\n");
for (int i = 0; i < blocks; i++) {
    scanf("%d", &blockSize[i]);
}

printf("Enter number of processes: ");
scanf("%d", &processes);

int processSize[processes];

printf("Enter sizes of processes:\n");
for (int i = 0; i < processes; i++) {
    scanf("%d", &processSize[i]);
}

firstFit(blockSize, blocks, processSize, processes);
bestFit(blockSize, blocks, processSize, processes);
worstFit(blockSize, blocks, processSize, processes);

return 0;
}

```

Output:

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.   Process Size   Block No.
1             212           2
2             417           5
3             112           3
4             426          Not Allocated

--- Best Fit ---
Process No.   Process Size   Block No.
1             212           4
2             417           2
3             112           3
4             426           5

--- Worst Fit ---
Process No.   Process Size   Block No.
1             212           5
2             417           2
3             112           4
4             426          Not Allocated

Process returned 0 (0x0)   execution time : 25.001 s
Press any key to continue.
```

Program -7

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

```
#include <stdio.h>
```

```
#define MAX 50
```

```
int checkHit(int frames[], int frameSize, int page) {  
    for (int i = 0; i < frameSize; i++)  
        if (frames[i] == page)  
            return 1;  
    return 0;  
}
```

```
void printFrames(int frames[], int frameSize) {  
    for (int i = 0; i < frameSize; i++) {  
        if (frames[i] == -1)  
            printf("- ");  
        else  
            printf("%d ", frames[i]);  
    }  
}
```

```
int fifo(int pages[], int n, int frameSize) {  
    int frames[10], index = 0, faults = 0;  
  
    for (int i = 0; i < frameSize; i++)  
        frames[i] = -1;  
  
    printf("\n\n===== FIFO WORKING =====\n");
```



```

for (int i = 0; i < n; i++) {
    printf("Page %d -> ", pages[i]);

    if (!checkHit(frames, frameSize, pages[i])) {
        frames[index] = pages[i];
        index = (index + 1) % frameSize;
        faults++;

        printf("FAULT | ");
        printFrames(frames, frameSize);
    } else {
        printf("HIT | ");
        for (int j = 0; j < frameSize; j++)
            printf("- ");
    }

    printf("\n");
}

return faults;
}

int lru(int pages[], int n, int frameSize) {
    int frames[10], time[10];
    int faults = 0, counter = 0;

    for (int i = 0; i < frameSize; i++) {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\n\n==== LRU WORKING =====\n");

    for (int i = 0; i < n; i++) {

```

```

int page = pages[i];
int hit = 0;

printf("Page %d -> ", page);

for (int j = 0; j < frameSize; j++) {
    if (frames[j] == page) {
        hit = 1;
        time[j] = ++counter;
        break;
    }
}

if (!hit) {
    int pos = 0;

    for (int j = 1; j < frameSize; j++) {
        if (time[j] < time[pos])
            pos = j;
    }

    frames[pos] = page;
    time[pos] = ++counter;
    faults++;

    printf("FAULT | ");
    printFrames(frames, frameSize);
} else {
    printf("HIT | ");
    for (int j = 0; j < frameSize; j++)
        printf("- ");
}

printf("\n");

```

```

    }

    return faults;
}

int optimal(int pages[], int n, int frameSize) {
    int frames[10];
    int faults = 0;

    for (int i = 0; i < frameSize; i++)
        frames[i] = -1;

    printf("\n\n===== OPTIMAL WORKING =====\n");

    for (int i = 0; i < n; i++) {

        printf("Page %d -> ", pages[i]);

        if (checkHit(frames, frameSize, pages[i])) {
            printf("HIT | ");
            for (int j = 0; j < frameSize; j++)
                printf("- ");
            printf("\n");
            continue;
        }

        int empty = -1;

        for (int j = 0; j < frameSize; j++) {
            if (frames[j] == -1) {
                empty = j;
                break;
            }
        }
    }
}

```

```

if (empty != -1) {
    frames[empty] = pages[i];
} else {
    int farthest = -1, replace = 0;

    for (int j = 0; j < frameSize; j++) {
        int k;
        for (k = i + 1; k < n; k++) {
            if (frames[j] == pages[k])
                break;
        }

        if (k > farthest) {
            farthest = k;
            replace = j;
        }
    }

    frames[replace] = pages[i];
}

faults++;

printf("FAULT | ");
printFrames(frames, frameSize);
printf("\n");
}

return faults;
}

int main() {
    int pages[MAX], n, frameSize;

```

```

int fifoF, lruF, optF;

printf("Enter number of pages: ");
scanf("%d", &n);

printf("Enter reference string:\n");
for (int i = 0; i < n; i++)
    scanf("%d", &pages[i]);

printf("Choose number of page frames (3 or 4): ");
scanf("%d", &frameSize);

if (frameSize != 3 && frameSize != 4) {
    printf("Invalid choice! Only 3 or 4 allowed.\n");
    return 0;
}

fifoF = fifo(pages, n, frameSize);
lruF = lru(pages, n, frameSize);
optF = optimal(pages, n, frameSize);

printf("\n\n===== FINAL RESULT =====\n");
printf("FIFO    : %d faults\n", fifoF);
printf("LRU     : %d faults\n", lruF);
printf("OPTIMAL  : %d faults\n", optF);

printf("\n\n===== BEST ALGORITHM =====\n");

if (fifoF <= lruF && fifoF <= optF)
    printf("FIFO is best\n");
else if (lruF <= fifoF && lruF <= optF)
    printf("LRU is best\n");
else
    printf("Optimal is best\n");

```

```

printf("\nFrames used: %d\n", frameSize);

return 0;
}

```

Output:

```

Enter number of pages: 12
Enter reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Choose number of page frames (3 or 4): 3

===== FIFO WORKING =====
Page 1 -> FAULT | 1 - -
Page 2 -> FAULT | 1 2 -
Page 3 -> FAULT | 1 2 3
Page 4 -> FAULT | 4 2 3
Page 1 -> FAULT | 4 1 3
Page 2 -> FAULT | 4 1 2
Page 5 -> FAULT | 5 1 2
Page 1 -> HIT   | - - -
Page 2 -> HIT   | - - -
Page 3 -> FAULT | 5 3 2
Page 4 -> FAULT | 5 3 4
Page 5 -> HIT   | - - -

===== LRU WORKING =====
Page 1 -> FAULT | 1 - -
Page 2 -> FAULT | 1 2 -
Page 3 -> FAULT | 1 2 3
Page 4 -> FAULT | 4 2 3
Page 1 -> FAULT | 4 1 3
Page 2 -> FAULT | 4 1 2
Page 5 -> FAULT | 5 1 2
Page 1 -> HIT   | - - -
Page 2 -> HIT   | - - -
Page 3 -> FAULT | 3 1 2
Page 4 -> FAULT | 3 4 2
Page 5 -> FAULT | 3 4 5

===== OPTIMAL WORKING =====
Page 1 -> FAULT | 1 - -
Page 2 -> FAULT | 1 2 -
Page 3 -> FAULT | 1 2 3
Page 4 -> FAULT | 1 2 4
Page 1 -> HIT   | - - -
Page 2 -> HIT   | - - -
Page 5 -> FAULT | 1 2 5
Page 1 -> HIT   | - - -
Page 2 -> HIT   | - - -
Page 3 -> FAULT | 3 2 5
Page 4 -> FAULT | 4 2 5
Page 5 -> HIT   | - - -

```